

Lab & Activities Week 01 LLM Fundamentals

Integrantes:

José Martínez-02220131016

Institución: Universidad de Santander

Pregrado: Ingeniería Se Sistemas

Docente: Jorge Galvis

25 de Agosto 2026

Activity 1

Activity 1 · Warm-up Concept Quiz

10 MIN

INDIVIDUAL

FORMATIVE

Answer from memory before checking — this is about surfacing what stuck from the theory block, not scoring.

1. What is a large language model fundamentally doing when it generates text?

- ☐ Looking up the most likely full sentence in a database of human writing
- ☐ Repeatedly predicting the single most probable next token given everything before it
- ☐ Translating the prompt into logic rules and executing them

Correct answer: B. Every visible capability (answering, coding, chatting) emerges from repeating next-token prediction.

2. As a rough rule of thumb, one token of English text is about how many characters?

- ☐ 1 character
- ☐ 10 characters
- ☐ 4 characters

Correct answer: C. 1 token $\approx$ 4 characters $\approx$ 0.75 words for English.

3. You're building a function that extracts a date from an email. Which setting best fits that task?

- ☐ Low temperature (e.g. 0–0.2)
- ☐ High temperature (e.g. 1.0+)
- ☒ Temperature has no effect on extraction tasks

Correct answer: A. Extraction has one right answer; low temperature keeps output deterministic and repeatable.

4. What does `top_p = 0.9` (nucleus sampling) do?

- ☐ Always picks exactly 9 candidate tokens
- ☐ Samples from the smallest set of tokens whose cumulative probability is at least 0.9
- ☐ Caps the output at 90% of max_tokens

Correct answer: B. It's a probability-mass cutoff, not a fixed count.

5. What does self-attention let a Transformer do?

- Compress the vocabulary into fewer tokens
- Store previous conversations permanently
- Let each token weigh how relevant every other token is when building its own representation

Correct answer: C. This is the mechanism behind resolving references like "it" to the right noun.

Check answers Score: 5 / 5

Activity 2

Follow along in week-01-llm-fundamentals.ipynb, sections 1–3:

1. Confirm your .env and API key work with a single test call.

1. Environment check

Load the API key and make one call to confirm everything is wired up.

```
import os
from dotenv import load_dotenv
from google import genai
from google.genai import types

load_dotenv()
assert os.environ.get("GOOGLE_API_KEY"), "Set GOOGLE_API_KEY in a .env file first."

client = genai.Client(api_key=os.environ["GOOGLE_API_KEY"])
MODEL = "gemini-flash-latest"

response = client.models.generate_content(
    model=MODEL,
    contents="In one sentence, what is a large language model?",
)
print(response.text)
```

[3] ✓ 5.6s Python

... A large language model is an artificial intelligence system trained on vast amounts of text data to understand

LLM Answer: *"A large language model is an artificial intelligence system trained on vast amounts of text data to understand, process, and generate human-like language."*

2. Run the temperature/top-p/top-k sweep together; each student calls out what they observe changing.

2. Sampling parameters: temperature, top-p, top-k

We'll send the **same prompt** several times, changing only `temperature`. Watch how the output goes from predictable to varied.

```
PROMPT = "Give me one creative name for a coffee shop. Reply with just the name."

for temp in [0.0, 0.4, 0.9, 1.5]:
    response = client.models.generate_content(
        model=MODEL,
        contents=PROMPT,
        config=types.GenerateContentConfig(temperature=temp, max_output_tokens=500),
    )
    print(f"temperature={temp:<4} -> {response.text}")
```

[9] ✓ 15.1s Python

... temperature=0.0 -> Morning Alchemy
temperature=0.4 -> Sonder & Steam
temperature=0.9 -> Fable & Froth
temperature=1.5 -> Perch & Pour

On this part of the Notebook I delete the function `.strip()` because they're throwing error's also y replace the value of the `max_output_tokens` to 500 to get the complete answer from the LLM

Run the cell above a few times. At `temperature=0.0` the answer should barely change between runs. At `temperature=1.5` you should see real variety (and occasionally something a little unhinged — that's expected).

Now let's isolate `top_p` and `top_k` by holding temperature fixed at a mid-range value.

```
for top_p in [0.1, 0.5, 0.95]:
    response = client.models.generate_content(
        model=MODEL,
        contents=PROMPT,
        config=types.GenerateContentConfig(temperature=0.9, top_p=top_p, max_output_tokens=500),
    )
    print(f"top_p={top_p:<5} -> {response.text}")

print()
for top_k in [1, 5, 40]:
    response = client.models.generate_content(
        model=MODEL,
        contents=PROMPT,
        config=types.GenerateContentConfig(temperature=0.9, top_k=top_k, max_output_tokens=500),
    )
    print(f"top_k={top_k:<3} -> {response.text}")
```

[10] ✓ 19.3s Python

... top_p=0.1 -> Fable & Foam
top_p=0.5 -> Roast & Reverie
top_p=0.95 -> Paper & Pour

top_k=1 -> Fable & Foam
top_k=5 -> Fable & Froth
top_k=40 -> Paper & Pour

On this part I do it removing the removal of the function `.strip` because it was throwing error's, also put the value or the `max_output_tokens` to 500 to get the complete answer from the LLM. I just see how the answer start's to get variations as the values of top k and top p increased

3. Run `count_tokens` on a few sentences of different lengths and compare against the "1 token $\approx$ 4 characters" rule of thumb.

For this part of the activity I add another phrase but in Spanish to see de difference between the tokenization in different languages, because the Spanish words have different tokenization heigh than the English words

The phrases was:

- "Hi!",
- "The capital of France is Paris.",
- "AI Agentic Engineering is a course about building autonomous systems with large language models, "
- "retrieval-augmented generation, and multi-agent orchestration frameworks like Google ADK and LangGraph.",
- "Una vez que hayas terminado de leer este texto, por favor, responde con un resumen en español. No olvides la puntuación y las reglas gramaticales.",



```
samples = [
    "Hi!",
    "The capital of France is Paris.",
    "AI Agentic Engineering is a course about building autonomous systems with large language models, "
    "retrieval-augmented generation, and multi-agent orchestration frameworks like Google ADK and LangGra
    "Una vez que hayas terminado de leer este texto, por favor, responde con un resumen en español. No ol
]

for text in samples:
    result = client.models.count_tokens(model=MODEL, contents=text)
    chars = len(text)
    tokens = result.total_tokens
    ratio = chars / tokens if tokens else 0
    print(f"chars={chars:<4} tokens={tokens:<4} chars/token={ratio:.2f} | {text[:50]!r}")
```

[3] ✓ 1.3s Python

chars	tokens	chars/token	text
3	3	1.00	'Hi!'
31	8	3.88	'The capital of France is Paris.'
200	36	5.56	'AI Agentic Engineering is a course about building '
146	33	4.42	'Una vez que hayas terminado de leer este texto, po'

The rule holds true well in simple medium-length sentences (Paris, ratio 3.88) the ideal case where the "1 token $\approx$ 4 characters" heuristic originated.

It breaks down strongly in short text ("Hi!", ratio 1.00). Short texts have overhead: each short word and punctuation mark ("!") consumes its own full token; there is no "dilution" in longer text. The 4-character rule assumes average performance in long texts but fails in short messages (short system prompts, commands, etc. important in real-world apps). It reverses in long technical text (ratio 5.56, above the rule). Common English words/repeated technical vocabulary ("Agentic", "orchestration", "framework") compress well because the BPE tokenizer already treats these subwords as frequent units, more efficient than the rule predicts. Spanish comes close but falls short (4.42, not far from 4.0) accents, the letter ñ, and compound words add some cost, but not as dramatically as one might expect (modern tokenizers handle multi-language well).

The rule "1 token $\approx$ 4 characters" is a valid approximation only for average English in medium-length texts. It degrades at extremes (very short) and varies by language/vocabulary domain — relevant for budgeting costs/context limits in production; do not rely on a fixed rule.

Activity 3

For this activity I just generate a 3 sentences with different temperture and top_p and mixed to get the 3 sentences in different order

```
import random

PROMPT_DETECTIVE = "Write a one-sentence opening for a daily story."

SETTINGS = [
    {"label": "low", "temperature": 0.0, "top_p": 0.1},
    {"label": "mid", "temperature": 0.7, "top_p": 0.9},
    {"label": "high", "temperature": 1.4, "top_p": 1.0},
]

shuffled = SETTINGS.copy()
random.shuffle(shuffled)

outputs = []
for cfg in shuffled:
    response = client.models.generate_content(
        model=MODEL,
        contents=PROMPT_DETECTIVE,
        config=types.GenerateContentConfig(
            temperature=cfg["temperature"], top_p=cfg["top_p"], max_output_tokens=500
        ),
    )
    outputs.append(response.text.strip())

for i, text in enumerate(outputs):
    print(f"Output {chr(65+i)}:\n{text}\n")

_answer_key = {chr(65 + i): shuffled[i] for i in range(len(shuffled))}
```

[4] ✓ 14.5s Python

... Output A:
:* "By the time the steam had cleared from the bathroom mirror, Arthur had already decided to live

Output B:
The morning smelled of rain and burnt toast, a familiar comfort that made the sudden silence from the street

Output C:
The world always felt full of quiet possibility in the ten minutes between the

Later I just ask for an AI to construct one algorithm that throw me the phrase with the temperture and top_p. Also this algorithm organize the dictionary and give me the answer like you see on the below image

```
Output C:
The world always felt full of quiet possibility in the ten minutes between the
```

```
# Reveal - run this only after you've written your ranking and justification above.
print("True settings, from most to least deterministic:")
for label in sorted(_answer_key, key=lambda l: (_answer_key[l]["temperature"], _answer_key[l]["top_p"])):
    cfg = _answer_key[label]
    print(f" Output {label}: temperature={cfg['temperature']}, top_p={cfg['top_p']} ({cfg['label']})")
```

[5] ✓ 0.0s Python

... True settings, from most to least deterministic:
Output C: temperature=0.0, top_p=0.1 (low)
Output A: temperature=0.7, top_p=0.9 (mid)
Output B: temperature=1.4, top_p=1.0 (high)

Generate + Code + Markdown

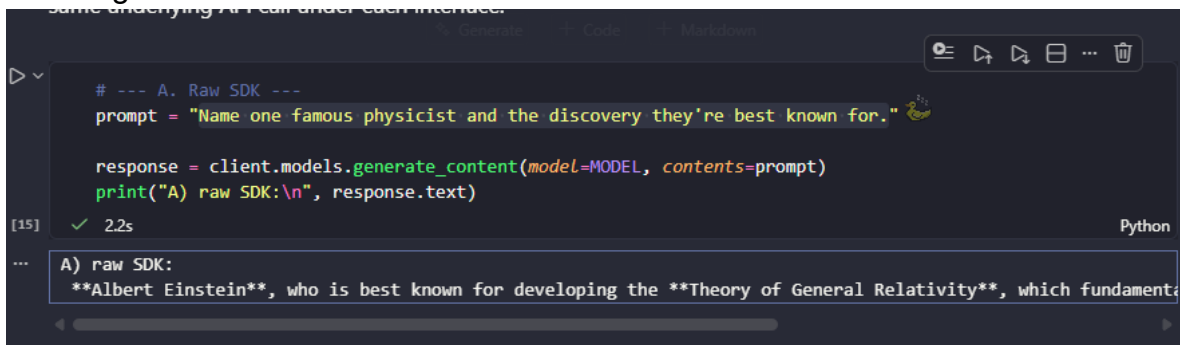
Output C (temp=0.0, top_p=0.1): Highly peaked distribution — the model almost always chooses the most probable token; sampling has very little room to vary. Generic/abstract text ("quiet possibility... between the") is a typical marker of low temperature: safe but bland.

Output A (temp=0.7, top_p=0.9): Moderately open distribution; the nucleus (top_p=0.9) allows quite a few candidates to pass, but not all. Result: coherent, with an odd touch (the stray colon at the beginning) a sign that sampling has already deviated slightly from the most probable path, but without breaking the structure.

Output B (temp=1.4, top_p=1.0): Flat distribution; top_p=1.0 doesn't clip anything; high temperature flattens the probabilities even further. More "poetic"/unexpected text (rare combination "rain and burnt toast... sudden silence"), less obvious juxtaposition a marker of almost unrestricted sampling.

Activity 4

For this part I just change the prompt because I just want know about physicist, the new prompt used was "Name one famous physicist and the discovery they're best known for." Later I execute all of the 3 different forms to ask at LLM like you see on the images below



The screenshot shows a Jupyter Notebook interface. The code cell contains the following Python code:

```
# --- A. Raw SDK ---
prompt = "Name one famous physicist and the discovery they're best known for."
response = client.models.generate_content(model=MODEL, contents=prompt)
print("A) raw SDK:\n", response.text)
```

The output cell shows the result of the code execution:

```
A) raw SDK:
**Albert Einstein**, who is best known for developing the Theory of General Relativity, which fundamenta
```

LLM Responses: "A) raw SDK:

*****Albert Einstein****, who is best known for developing the **Theory of General Relativity**, which fundamentally changed our understanding of gravity, space, and time."*

```

# --- B. Google ADK (minimal agent) ---
import asyncio
from google.adk.agents import Agent
from google.adk.runners import InMemoryRunner

adk_agent = Agent(
    model=MODEL,
    name="week1_demo_agent",
    instruction="Answer concisely, in 1-2 sentences.",
)

async def ask_adk_agent(agent, prompt, app_name="week1_app", user_id="student"):
    """Small reusable helper: runs one prompt through an ADK agent and returns the final text."""
    runner = InMemoryRunner(agent=agent, app_name=app_name)
    session = await runner.session_service.create_session(app_name=app_name, user_id=user_id)
    content = types.Content(role="user", parts=[types.Part.from_text(text=prompt)])
    final_text = None
    for event in runner.run(user_id=user_id, session_id=session.id, new_message=content):
        if event.content and event.content.parts and event.content.parts[0].text:
            final_text = event.content.parts[0].text
    return final_text 🐼

print("B) Google ADK agent:\n", await ask_adk_agent(adk_agent, prompt))

```

✓ 3.8s Python

B) Google ADK agent:
 Albert Einstein is best known for developing the theory of relativity, which fundamentally revolutionized our understanding of space, time, and gravity."

LLM Responses: "B) Google ADK agent:

Albert Einstein is best known for developing the theory of relativity, which fundamentally revolutionized our understanding of space, time, and gravity."

```

# --- C. LangChain chat model ---
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(model=MODEL)
lc_response = llm.invoke(prompt)
print("C) LangChain:\n", lc_response.content)

```

[17] ✓ 8.3s Python

C) LangChain:
 [{"type": "text", "text": "***Albert Einstein** is best known for developing the **Theory of General Relativity**", "signature": "ErMMCrAMARFNMg8rrfCa0y7x3oU+MFWY4YeHRU8YVjvvdBbXZr/KmH2KHPSaDWEpo6fVSOE4N80QiuDxlrRq4ptbY....."}]

LLM Responses: "C) LangChain:

*[{"type": "text", "text": "***Albert Einstein** is best known for developing the **Theory of General Relativity**", "signature": "ErMMCrAMARFNMg8rrfCa0y7x3oU+MFWY4YeHRU8YVjvvdBbXZr/KmH2KHPSaDWEpo6fVSOE4N80QiuDxlrRq4ptbY....."}]*

A (raw SDK): returns direct plain text, `response.text` — the rawest form, without extra layers.

B (Google ADK): passes through the Agent with the instruction "Answer concisely, in 1-2 sentences." — this system instruction truncates and forces concise formatting, which is why it's shorter than A (which didn't have an instruction, leaving the model to add Markdown/bold/formula formatting).

C (LangChain): same model, but ChatGoogleGenerativeAI wraps the response in a block list structure (`[{'type': 'text', 'text': ..., 'extras': {'signature': ...}}]`) instead of a plain string — the signature is internal Gemini metadata (thought signature), not part of the actual text. The background content is equivalent to A; only the library's wrapper/format changes.

Conclusion: Same Gemini model underneath the 3, differences come from how each layer (raw SDK vs agent with instruction vs LangChain wrapper) post-processes/formats the output, not from the model itself.

I just created a github repository to upload all of the different sessions that I'm going to be working on. This is the URL: https://github.com/JoseMartinez2611/ai_agentic_class

Activity 6

Corte 1 Project Checkpoint & Independent Work

- Choose the document domain your assistant will answer questions about (e.g. your program's course catalog, a hobby's rulebook, your own class notes, a product's documentation). Pick something you have 5–20 real documents or pages for — you'll need it for chunking in Week 4.

R/ A calculus tutoring assistant that answers questions about a first-semester calculus course (limits, derivatives, and integrals). It solves exercises step by step, explaining the reasoning and the theorem/rule behind each step, and answers theoretical questions grounded in calculus lecture notes and textbook chapters via RAG, rather than relying only on the model's general knowledge.

- Write a 3–5 sentence project proposal (what it answers questions about, who it's for) and push it as PROPOSAL.md to your course GitHub repository.

R/ <https://github.com/JulianGuardian/Cubik-Lite>

